

**VERIFICATION OF HARDWARE IMPLEMENTATION OF SIMON-CIPHER
ENCRYPTION**

A Thesis submitted to the faculty of
San Francisco State University
In partial fulfillment of
the requirements for
the Degree

Master of Science

In

Engineering: Embedded Electrical and Computer Systems

by

Arohi Shivlal Vekariya

San Francisco, California

Fall 2021

Copyright by
Arohi Shivlal Vekariya
2021

Certification of Approval

I certify that I have read VERIFICATION OF HARDWARE IMPLEMENTATION OF SIMON-CIPHER ENCRYPTION by Arohi Shivilal Vekariya, and that in my opinion this work meets the criteria for approving a thesis submitted in partial fulfillment of the requirement for the degree Master of Science in Embedded Electrical and Computer Systems at San Francisco State University.

Dr. Hamid Mahmoodi Ph.D.

Professor of Electrical/ Computer Engineering,
Thesis Committee Chair

Dr. Xiaorong Zhang Ph.D.

Associate Professor of Computer Engineering

VERIFICATION OF HARDWARE IMPLEMENTATION OF SIMON-CIPHER ENCRYPTION

Arohi Shivlal Vekariya
San Francisco, California
2021

Data security, authentication and piracy prevention is extremely important in today's world as the threat towards data communication are increasing tremendously. To overcome security and cyber safety problems and maintaining flexibility, the U.S. National Security Agency came up with Simon and Speck family that is compatible with a diverse collection of Internet of Things (IOT) devices. Simon is a group of lightweight block Ciphers greatly in use for hardware implementation because of its performance, Speck on the other hand provides optimal performance in software implementations. A block Cipher in cryptography means an algorithm which works on fixed length groups of bits and used for large amount of data encryption along with exchange protocols. Simon and Speck support different block-sizes which helps increase the efficiency to encrypt comparatively large messages. The report focuses on verification of the Simon block using a coverage driven, class-based testbench to

ensure the stability of the design to be compatible with different block sizes and key lengths.

Acknowledgements

I am extremely grateful to my supervisor Dr. Hamid Mahmoodi for continuous support, remarks and patience throughout my master's research and applied project. I could not have accomplished this without his encouragement and advice. I would like to extend my sincere thanks to Dr. Xiaorong Zhang for being a committee member and assisting me through the process. Finally, I would like to share gratitude towards my friends and family for their affection and immense support which has helped me complete my journey of master's.

Table of Contents

List of Tables	viii
List of Figures.....	ix
1. Introduction.....	1
2. Importance of Lightweight Block Ciphers	4
3. The Simon Cipher Design	6
3.1 The Round Function	6
3.2 The Key Expansion Function.....	8
4. Hardware Implementation of Simon Cipher	11
5. Verification of Simon Cipher Design	13
5.1 Transaction class.....	13
5.2 Generator class.....	14
5.3 Driver class	15
5.4 Monitor class.....	16
5.5 Environment class.....	17
5.6 Interface class.....	18
5.7 Scoreboard class.....	19
5.8 Coverage class	21
5.9 Top module	22
5.10 Test program	23
6. Simulation Result.....	24
6.1 Functional Coverage and Coverage Results	25
Conclusion	31
References.....	32

List of Tables

Table 1. Performance Comparison in Hardware and Software Implementation	3
Table 2. Simon and Speck Parameters	5
Table 3. Simon block options	8

List of Figures

Figure 1. Round function based on Feistel network	7
Figure 2. Key expansion function	10
Figure 3. Hardware Implementation of Round function	11
Figure 4. Hardware implementation of Key Expansion.....	12
Figure 5. Transaction class.....	13
Figure 6. Generator class	14
Figure 7. Driver class.....	15
Figure 8. Monitor class	16
Figure 9. Environment class.....	17
Figure 10. Interface class.....	18
Figure 11. Scoreboard class	20
Figure 12. Scoreboard class contd.	20
Figure 13. Covergroup.....	22
Figure 14. Top module.....	22
Figure 15. Test.....	23
Figure 16. Simulation output	24
Figure 17. Simulation waveform	25
Figure 18. Coverage Report for 64 bins.....	25
Figure 19. Coverage Report for 128 bins.....	26
Figure 20. Coverage Report for 512 bins.....	28

Figure 21. Coverage Report for 1024 bins.....	28
Figure 22. Coverage report Comparison	30

1. Introduction

In today's world, security of data while communicating is one of the major concerns as the number of cyber-attacks is increasing greatly. Amongst numerous cryptographic algorithms established for encrypting messages, most of them were designed with a concern of satisfying computing requirements [4]. These encryption technologies were not very well compliant to the upcoming challenges in the field of data communication, especially for highly restricted devices. As is the technology developing to secure the data transfers, so is the skillset of hackers increasing making it difficult to obtain highest level of security. Lightweight cryptography is identified to stand against major security issues for highly constrained devices such as those used in government organizations, military, etc. The Simon and Speck algorithms were built with the goal of providing highest level of security and flexible with different sizes.

Encryption Algorithm are mainly divided into two sub-categories viz. Symmetric encryption and Asymmetric Encryption. As the name suggests, symmetric encryption uses same key for both encryption as well as decryption, whereas asymmetric encryption has unique key for encryption and decryption, i.e., public key and private key respectively. Further ahead, symmetric encryption is divided into block type (e.g., Advanced Encryption Standard) and stream cipher type (e.g., the one-time pad). Advanced Encryption Standard (AES) is one of the most famous encryption methodologies that has replaced the Data Encryption Standard (DES). AES is compatible with 128-, 192- or 256-bits key. Despite being an efficient algorithm, AES had some drawbacks such as:

- i. The encryption algorithm for all block sizes remains the same, hence making it easier to hack.
- ii. Software implementation of AES algorithm is difficult, considering different factors such as performance and security.
- iii. Due to lack of availability of area or power, AES engines are not suitable for resource-oriented platforms.

Considering, these factors in mind and to overcome the above-mentioned drawbacks SIMON and SPECK were developed by National Security Agency (NSA) in July 2013 [\[1\]](#) providing great flexibility and high performance in both hardware as well as software implementations [\[3\]](#). Before discussing the compatibility and performance of Simon-Cipher, one aspect to be considered is about block cipher. A block cipher is unable to provide security by itself. Based upon the security and size requirement of the application, algorithms are developed utilizing the flexibility of the block cipher.

Table1 supports the fact that SIMON and SPECK provide best performance in hardware as well as software implementations [\[2\]\[4\]](#), but when it comes to optimal performance, Simon is the most suitable for Hardware implementations (such as tags and RFID readers) whereas Speck is optimal for Software implementation (such as RFID read data), Both the families show compatibility with

Table 1. Performance Comparison in Hardware and Software Implementation

Family Name and Size (2n/nm)	Hardware Area (GE)	Hardware Throughput (kbps)	Software Flash (bytes)	Software Throughput (kbps)
Simon 48/96	763	15.0	196	589
Speck 48/96	884	12.0	132	943
Simon 64/96	838	17.8	274	540
Speck 64/96	984	14.5	182	888
Simon 64/128	1000	16.7	282	515
Speck 64/128	1127	13.8	186	855
Simon 96/69	984	14.8	454	454
Speck 96/96	1134	13.8	276	866
Simon 128/128	1317	22.9	732	342
Speck 128/128	1396	12.1	396	768

a range of different block size and key size. The data shows that at least 12-kbps throughput is achieved at 100kHz for hardware. It varies with respect to area and size of the block cipher being used.

2. Importance of Lightweight Block Ciphers

Lightweight ciphers are essential for a safe and secure communication while operating on a resource-constrained system. Some of the major features of a lightweight block cipher that draws attention towards itself are less power consumption, occupying less memory, higher throughput, strong immunity against various cyber attacks and many more. While developing the Simon cipher, there were certain requirements kept in mind as follows:

- i) Achieve highest security against standard Chosen plaintext attacks (CPA) and chosen ciphertext attacks (CCA).
- ii) Immunity against related-key attacks.
- iii) Comparatively smaller area for specified throughput.
- iv) Flexible design compatible with different key size and block size.

“Lightweight” itself is used to state that the algorithm works well with some restrained systems. In the process of developing an algorithm that could satisfy most of the above stated requirements, it became difficult to let go of any requirement and hence, Simon and Speck were both developed considering both software as well as hardware implementations.

A deterministic algorithm functioning on a given input block of plaintext with a symmetric key is a block cipher. A ciphertext is the encrypted data then produced from the plaintext. The plaintext block is a group of bits of fixed length and one block is served at a time unlike the stream cipher type where encryption is performed bitwise on the data. Available lightweight algorithms have limitations for the flexibility they can support in terms of compatibility with complex constrained systems, block size and key size supported and many more. Serialization is one of the

important factors that needs to be considered while implementing a system, which is divided into two types, viz. Fully serialized or bit-serial, and Un-serialized or iterated [\[2\]](#). A fully serialized implementation is one that updates single bit at a time and the un-serialized one updates entire block during each cycle. S-box or substitution box are used in block ciphers to cast a shadow over association of key and ciphertext which is mainly used in AES. Optimization of encryption algorithms is improvised with their ability to be serialized adeptly.

The Simon and Speck algorithm support different block sizes and key sizes as shown in Table2.

Table 2. Simon and Speck Parameters

Block Size	Key Size
32	64
48	72, 96
64	96, 128
96	96, 144
128	128, 192, 256

The Simon and Speck families are flexible enough to support different block sizes, 32, 48, 64, 96, 128 each of which supports up to 3 different key sizes.

3. The Simon Cipher Design

The Simon Cipher design for encryption mainly comprises of two different functions, the Round function, and the Key Expansion function. Simon block is represented as $\text{Simon}_{2n/nm}$, where n is the word size and m is the number of key word which is discussed in Table 3. The design considered for verification has a word size of 16 and the corresponding key words are 4, hence the block is $\text{Simon}_{32/64}$ having a plaintext block of 32 bit and the symmetric key of 64bit. Due to high speed and strong immunity against traditional attacks, hardware encryption is preferable.

3.1 The Round Function

The round function is based on a balanced Feistel network, which is a symmetric structure named after German-born physicist and cryptographer *Horst Feistel* [8]. One of the advantages of using the Feistel network in Simon Cipher encryption is that the process of encryption and decryption are symmetric and includes the Round function which runs for a specified number of times. The round function is constructed using following three functions:

- left circular shift, S^j by j bits, ($j = 1, 2$, or 8)
- bitwise AND,
- bitwise XOR, $\oplus$.

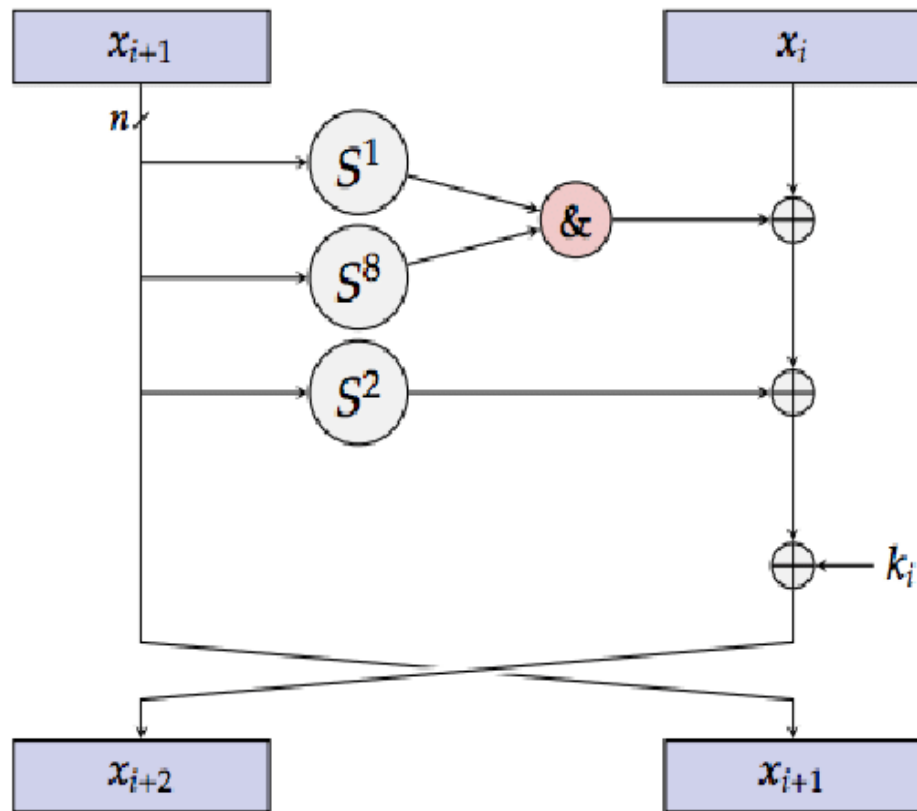
The round function is a two stage Feistel-map, R_k , which can be mathematically stated as:

$$R_k(x, y) = (y \oplus (S^1x \& S^8x) \oplus S^2x \oplus k, x) \quad (1)$$

where the round key $f(x) = (S^1x \& S^8x) \oplus S^2x$ and k is reversed to perform decryption of the ciphertext.

The functioning of the Round function is depicted in Figure 1. The x_{i+1} and x_i are the upper and lower words respectively of a block which is receiving its initial values from the plaintext, which is the data to be encrypted.

Figure 1. Round function based on Feistel network



The x_{i+1} undergoes a left circular shift by 1 bit and by 8 bits and both the shifted values are then ANDed. The x_i is XORed with the ANDed output and passed on further where it undergoes bitwise XOR with the left circular shifted x_{i+1} by 2 bits. Lastly, the value is XORed with the key value k_i and the outputs are swapped providing the sub-ciphers. Table 3 provides reference for the sizes of key and corresponding key words for different Simon blocks. Whitening steps, which

Table 3. Simon block options

Block Size (2n)	Key Size (mn)	Word Size (n)	Key words (m)	Rounds (T)	Constant Seq
32	64	16	4	32	z0
48	72	24	3	36	z0
	96		4	36	z1
64	96	32	3	42	z2
	128		4	44	z3
96	96	48	2	52	z2
	144		3	54	z3
128	128	64	2	68	z2
	192		3	69	z3
	256		4	72	z4

can increase security but at the same time, affects the circuit size and since Simon is the most suitable for Hardware implementations, SIMON design does not include plaintext and ciphertext whitening.

3.2 The Key Expansion Function

The Simon block requires a unique key of respective size at every round which is generated using the key expansion function as described in [figure 2](#). The constant sequences z0 to z5 are generated by Linear Feedback Shift Register (LFSR) using the below given sequences u, v and w [\[2\]](#):

$u = 1111101000100101011000011100110...$

$v = 1000111011111001001100001011010...$

$w = 1000010010110011111000110111010...$

$z_0 = u$

$z_1 = v$

z_2, z_3 and z_4 are generated by XORing u, v , and w with period 2 sequence $t = 010101...$ Hence, the constant sequence values are as follows:

$z_0 = 11111010001001010110000111001101111101000100101011000011100110...$

$z_1 = 10001110111110010011000010110101000111011111001001100001011010...$

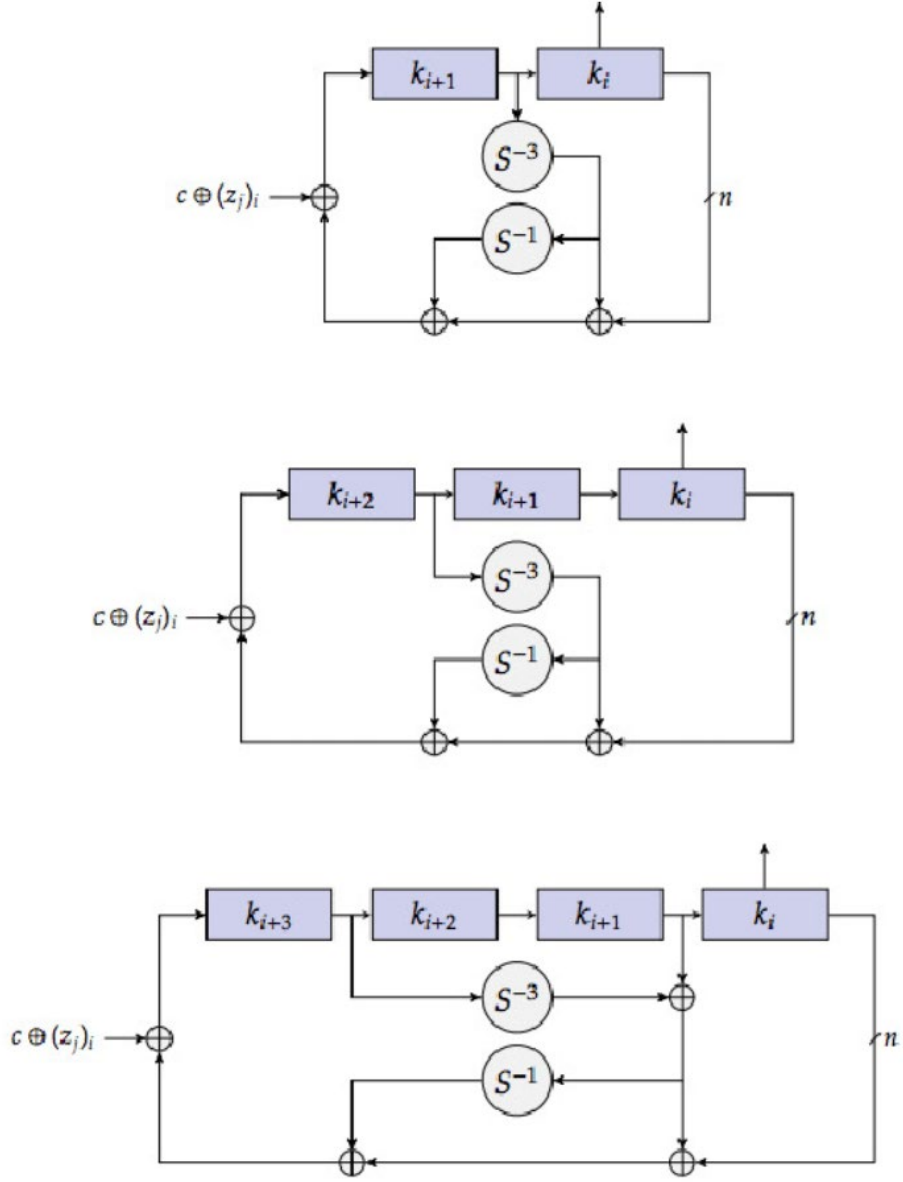
$z_2 = 1010111101110000001101001001100 0101000010001111110010110110011...$

$z_3 = 1101101110101100011001011110000 0010010001010011100110100001111...$

$z_4 = 1101000111100110101101100010000 0010111000011001010010011101111...$

The round key generation is defined in (2) for different values of key words being 2, 3, or 4, from top.

Figure 2. Key expansion function



$$k_{i+m} = \begin{cases} c \oplus (z_j)_i \oplus k_i \oplus (I \oplus S^{-1})S^{-3}k_{i+1}, \\ c \oplus (z_j)_i \oplus k_i \oplus (I \oplus S^{-1})S^{-3}k_{i+2}, \\ c \oplus (z_j)_i \oplus k_i \oplus (I \oplus S^{-1})(S^{-3}k_{i+3} \oplus k_{i+1}), \end{cases} \quad (2)$$

for $0 \leq i \leq T-m$ where i is the number of rounds of iteration.

4. Hardware Implementation of Simon Cipher

The hardware implementation of Simon Cipher block is depicted in Figure 2 [6] and Figure 3 [7]. n and m are word size and respective key words which can be referred to from [Table 1](#). The circuit is the implementation of the Round function discussed in [Section 3.1](#). Multiplexers and logic gates are used for the implementation of the same.

Figure 3. Hardware Implementation of Round function

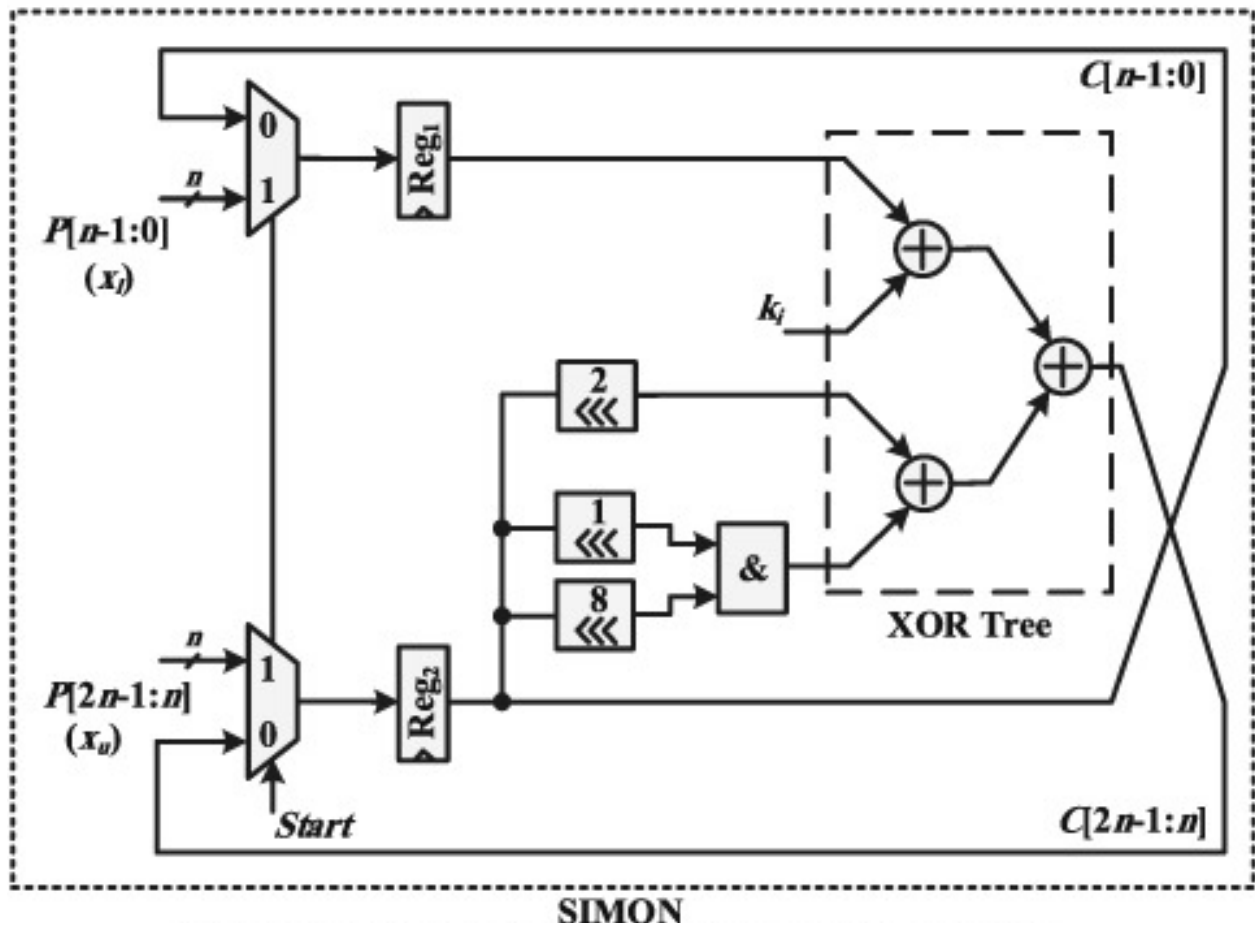


Figure 4. Hardware implementation of Key Expansion

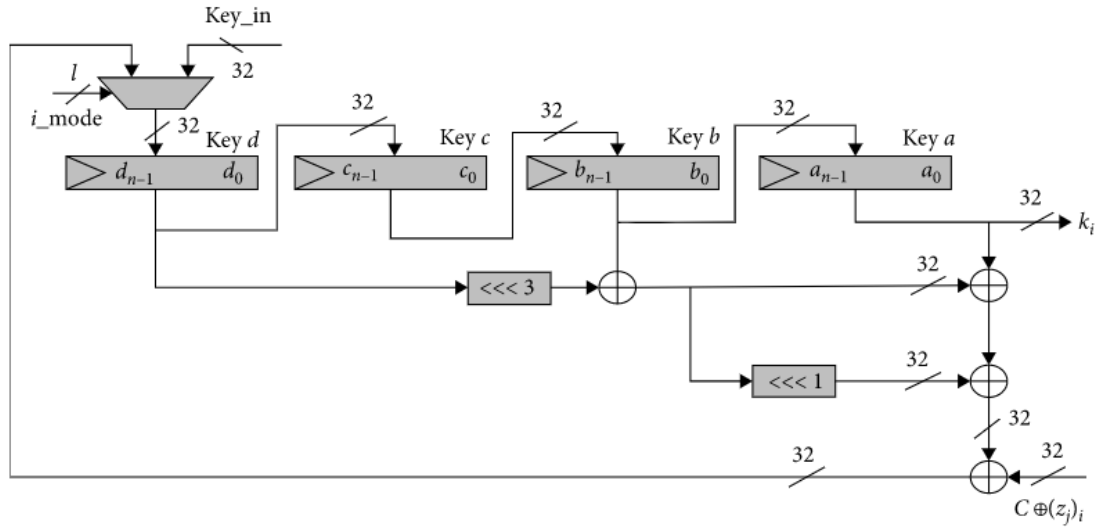


Figure 3 depicts the round function of the Simon Cipher algorithm wherein the input value x gets divided into two parts of x_{lower} and x_{upper} . These values then undergo multiple bitwise XOR, bitwise AND, and left circular shifts. The cycle goes on for $T = 32$ rounds, i.e., at the end of 31st cycle, the output achieved from the round function will be the expected ciphertext. Figure 4 is the hardware implementation of the block diagram of key expansion shown in figure 2. There are three block diagrams of which the hardware implementation refers to the last block diagram for $m = 4$. The implementation comprises of 64 bits key which is divided into 4 parts key a, key b, key c, key d. The key expansion performs the task of converting the key to round key to make sure that the Simon module receives new value of key and shifts the values circularly, which is clear from the simulation result and waveform.

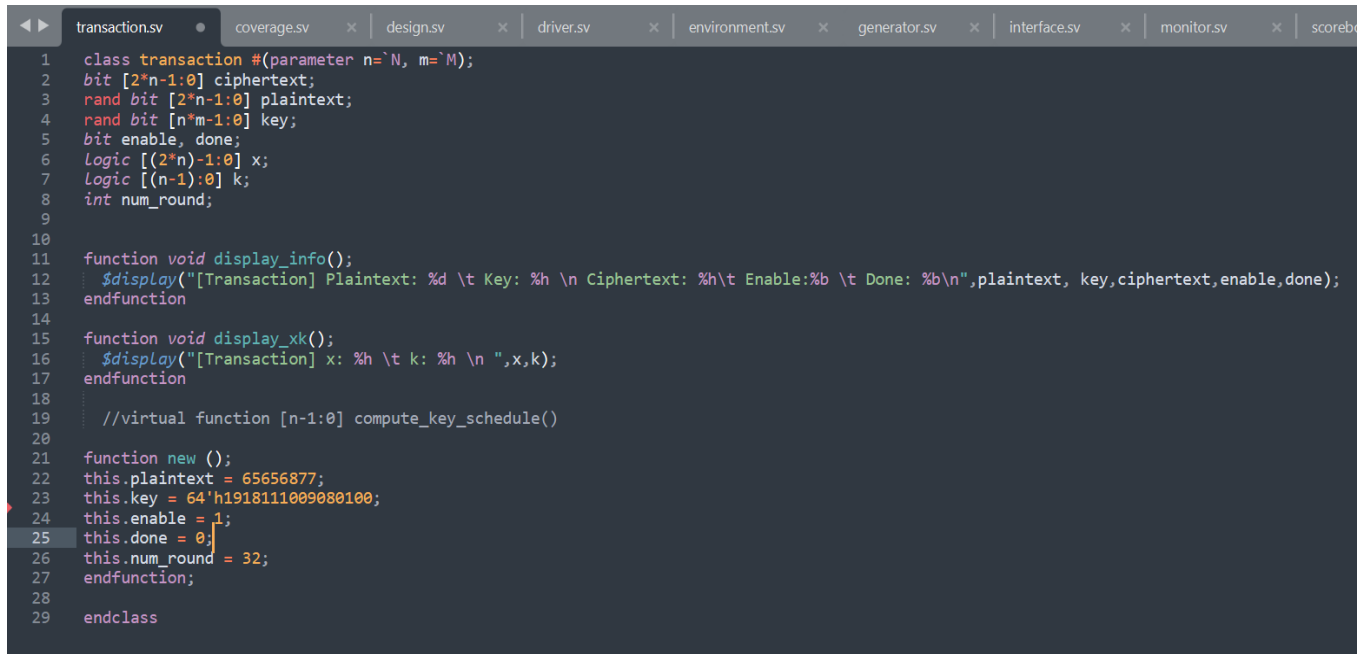
5. Verification of Simon Cipher Design

The verification plan for the Simon Cipher design is a class based testbench in System Verilog that comprises of self-checking components as well as covers corner points such as the plaintext should not be generated before the 32nd cycle, the done signal goes high at the 32nd cycle when the ciphertext is produced and that is when the plaintext should be generated. The testbench comprises of different classes making it a self-checking testbench that compares and shows the expected and actual output.

5.1 Transaction class

The transaction class comprises of randomized declaration of the input and output ports. Constraints specifying the range of possible values of randomized input is specified along with the initial values if any.

Figure 5. Transaction class



```

1  class transaction #(parameter n=`N, m=`M);
2  bit [2*n-1:0] ciphertext;
3  rand bit [2*n-1:0] plaintext;
4  rand bit [n*m-1:0] key;
5  bit enable, done;
6  logic [(2*n)-1:0] x;
7  logic [(n-1):0] k;
8  int num_round;
9
10
11 function void display_info();
12     $display("[Transaction] Plaintext: %d \t Key: %h \n Ciphertext: %h\t Enable:%b \t Done: %b\n",plaintext, key,ciphertext,enable,done);
13 endfunction
14
15 function void display_xk();
16     $display("[Transaction] x: %h \t k: %h \n ",x,k);
17 endfunction
18
19 //virtual function [n-1:0] compute_key_schedule()
20
21 function new ();
22     this.plaintext = 65656877;
23     this.key = 64'h1918111009080100;
24     this.enable = 1;
25     this.done = 0;
26     this.num_round = 32;
27 endfunction;
28
29 endclass

```

5.2 Generator class

The Generator class generates stimulus for the ports defined inside the transaction class. The randomization of ports of transaction class is performed and checked inside the generator class. The generated stimulus is then sent to Driver class via mailbox. A mailbox is used to exchange the data between classes.

Figure 6. Generator class

```
//Generator Class
`timescale 1ns / 1ps
class generator;

    //declaring transaction class
    transaction trans;

    //declaring mailbox
    mailbox gen2driv;

    //repeat count, to specify number of itms to generate
    int repeat_count;
    event gen_end;

    //event
    event ended;

    virtual simon_interface simon_vif;

    //constructor
    function new( virtual simon_interface simon_vif_new, mailbox gen2driv, event ended, int repeat_cnt );
        //getting the mailbox handle from env
        this.gen2driv = gen2driv;
        this.ended = ended;
        this.trans = new();
        this.repeat_count = repeat_cnt;
        this.simon_vif = simon_vif_new;
    endfunction

    //main task generates(creat and randomizes) the packets and puts into ailbox
    task main();
        $display("-----Begin Generator-----");
        repeat(repeat_count) begin
            trans = new();
            if(!trans.randomize())$fatal("Gen:: trans randomization failed");

            $display("Generated Transaction...");
            trans.display_info;
            gen2driv.put(this.trans);
        end
        $display("repeat count: %d", repeat_count);
        ->ended;
    endtask
endclass
```


5.3 Driver class

The stimulus generated by Generator class is received in the Driver class via mailbox. The Driver class translates the random generated inputs into actual inputs based upon the specifications in the transaction class for the Design under test (DUT).

Figure 7. Driver class

```
//DRIVER class
`define DRIV_IF simon_vif.DRIVER.driver_cb
class simon_driver;

virtual simon_interface simon_vif;
mailbox gen2driv;
transaction trans;
event mon_done;
int count;

function new (virtual simon_interface simon_if, mailbox gen2driv_box, event mon_done, int repeat_count);
this.simon_vif = simon_if;
this.gen2driv = gen2driv_box;
this.mon_done = mon_done;
this.count = repeat_count;
endfunction

task main;
//gen2driv.get(trans);
$display("-----Begin Driver-----");
repeat(count) begin
gen2driv.get(trans);
$display("Driving Following Transaction...");
trans.display_info;
@(posedge simon_vif.DRIVER.clk);
if(trans.enable) begin
`DRIV_IF.en <= trans.enable;
`DRIV_IF.key <= trans.key;
`DRIV_IF.plaintext <= trans.plaintext;
//@(posedge simon_vif.DRIVER.clk);
end
$display("-----");
@(`DRIV_IF);
`DRIV_IF.en <= 0;
/*
while(`DRIV_IF.done != 1) begin
@(`DRIV_IF);
end
*/
wait(mon_done.triggered);
end

endtask
endclass
```

5.4 Monitor class

The Monitor class samples the interface signals within a task and sends the sampled packets to scoreboard where comparison of output values takes place.

Figure 8. Monitor class

```
//Monitor class
`timescale 1ns / 1ps
`define MON_IF simon_vif.MONITOR.monitor_cb
class monitor;

    //creating virtual interface handle
    virtual simon_interface simon_vif;

    //creating mailbox handle
    mailbox mon2scb;
    transaction trans;
    event mon_done;
    int count;

    //constructor
    function new(virtual simon_interface simon_vif, mailbox mon2scb, event mon_done, int repeat_count);
        //getting the interface
        this.simon_vif = simon_vif;
        //getting the mailbox handles from environment
        this.mon2scb = mon2scb;
        this.mon_done = mon_done;
        this.count = repeat_count;
    endfunction

    //Samples the interface signal and send the sample packet to scoreboard
    task main;
        repeat(count) begin
            trans = new();
            @(posedge `MON_IF.done);
            @(posedge simon_vif.MONITOR.clk);
            trans.plaintext = `MON_IF.plaintext;
            trans.key = `MON_IF.key;
            trans.done = `MON_IF.done;
            trans.enable = `MON_IF.en;
            trans.ciphertext = `MON_IF.ciphertext;

            @(posedge simon_vif.MONITOR.clk);
            if(`MON_IF.done) begin
                $display("Monitor Begin");
                $display("-----");
                trans.display_info;
                mon2scb.put(trans);
                end
                ->this.mon_done;
            end
        endtask
    endclass
```

5.5 Environment class

The environment class comprises of all the instances of multiple verification classes such as monitor, generator, scoreboard, and driver. The environment class connects all the classes and calls the main function from all the classes.

Figure 9. Environment class

```
//Environment class
class environment;

    generator gen;
    simon_driver drv;
    monitor mon;
    scoreboard scb;

    mailbox gen2drv;
    mailbox mon2sb;

    event gen_ended;
    event mon_done;
    virtual simon_interface simon_vif;

    function new (virtual simon_interface simon_if_new);
    this.simon_vif = simon_if_new;
    this.gen2drv = new();
    this.mon2sb = new();
    endfunction

    function void build();
        gen = new(simon_vif,gen2drv,gen_ended,450);
        drv = new(simon_vif,gen2drv,mon_done,450);
        mon = new(simon_vif,mon2sb, mon_done,450);
        scb = new(gen2drv,mon2sb, mon_done,450);
    endfunction

    task start;
        fork
        gen.main;
        drv.main;
        mon.main;
        scb.main;
        join
        //mon.main;
    endtask

    task run;
        build;
        start;
    endtask

endclass
```

5.6 Interface class

The interface class comprises of modports that define the directions of the ports for the design and the clocking blocks for Monitor and Driver.

Figure 10. Interface class

```

//Interface class
`timescale 1ns / 1ps
`include "utility.vh"
interface simon_interface #(parameter n=`N, m=`M)(input bit clk);

    bit rst, en, done;
    bit [2*n-1:0] plaintext;
    bit [n*m-1:0] key;
    reg [2*n-1:0] ciphertext;
    reg [(2*n)-1:0] x;
    reg [(n-1):0] k;
    reg [n-1:0] key_schedule;

    clocking cb @(posedge clk);
        output rst, en;
        output plaintext;
        output key;
        input done;
        input ciphertext;
    endclocking;

    clocking driver_cb @(posedge clk);
        default input #1 output #1;
        output en;
        output plaintext;
        output key;
        input ciphertext;
        input done;
        output k,x;
    endclocking

    clocking monitor_cb @(posedge clk);
        default input #1 output #1;
        input en;
        input plaintext;
        input key;
        input ciphertext;
        input done;
    endclocking

    modport DRIVER (clocking driver_cb,input clk, output rst);

    modport MONITOR (clocking monitor_cb,input clk,rst);

    function void display_info();
    $display("[Interface Received Data]\tPlaintext: %h \t Key: %h\n",plaintext, key);
    endfunction
endinterface

```

5.7 Scoreboard class

The scoreboard class comprises of a module that generates the output for the testbench which is referred as expected output. The expected output is compared with the output from the DUT and displays the message of pass or fail based upon the outputs.

Figure 11. Scoreboard class

```

include "utility.vh"
class scoreboard #(parameter n=N, m=M);

    transaction trans;
    mailbox driv2sb;
    mailbox mon2scb;
    int no_transaction;
    event mon_done;

    // Computation Variables
    bit [n*m-1:0] key;
    bit [2*n-1:0] plaintext;
    bit [n-1:0] round_key;
    bit [(2*n)-1:0] x;
    bit [2*n-1:0] next_x, y;
    bit [(n-1):0] k;
    bit [7:0] current_counter;
    bit [n-1:0] key_schedule [32-1:0];
    bit [n*m-1:0] key_exp_in;

    covergroup simon_coverage;
    PLAINTEXT: coverpoint trans.plaintext {option.auto_bin_max = 64;}
    KEY: coverpoint trans.key {option.auto_bin_max = 64;}
    endgroup

    function new(mailbox driv2sb, mailbox mon2scb, event mon_done, int repeat_count);
        this.driv2sb = driv2sb;
        this.mon2scb = mon2scb;
        this.mon_done = mon_done;
        this.no_transaction = repeat_count;
        simon_coverage = new();
    endfunction

    function bit [(2*n)-1:0] compute_cipher (
        input bit [(2*n)-1:0] x,
        input bit [(n-1):0] k, ref bit [(2*n)-1:0] y);

        bit [n-1:0] x_0, x_1, rol_1, rol_8, rol_2;
        x_0 = x[n-1:0];
        x_1 = x[(2*n)-1:n]; // 31:16 -> 31:15
        y[n-1:0] = x_1;
        rol_1 = {x_1[0+: (n-1)], x_1[n-1]};
        rol_2 = {x_1[0+: (n-2)], x_1[n-1:2]};
        rol_8 = {x_1[0+: (n-8)], x_1[n-1:8]};

        y[(2*n)-1:n] = (((rol_1 & rol_8) ^ x_0) ^ rol_2) ^ k;
        return y;
    endfunction

    function bit [n-1:0] compute_round_key(
        bit [n*m-1:0] key, bit [6:0] i,
        ref bit [n-1:0] key_i);

        bit [n-1:0] tmp, tmp2, tmp3, ror3, ror1;
        bit [0:61] z;
        logic [7:0] index;
        z = 62'b1111101000100101011000011100110111101000100101011000011100110;
        if(i<m)
            key_i = key[(n*(i+1))-1:n];
        else begin
            tmp = {key[(n*(m-1))+3], key[n*m-1-(n-3)]};
            ror3 = tmp;
            if (m==4) begin
                tmp2 = tmp ^ key[(2*n)-1:n];
                tmp = tmp2;
            end
            ror1 = tmp ^ {tmp[0], tmp[n-1:1]};
            tmp=ror1;
            index = (i-m)<62? i-m: (i-m)-62;
            key_i = ~key[n-1:0] ^ tmp ^ z[index] ^ 2'b11;
        end
    endfunction

```

Figure 12. Scoreboard class contd.

```

task setup_keySchedule(ref bit [n-1:0] round_key, ref bit [(n-1):0] k);
    for(int i = 0; i < m; i++) begin
        compute_round_key(key_exp_in, (current_counter), round_key);
        this.k = this.key[(current_counter+1)*(n)-1:n];
        key_schedule[i] = key[(n*(i+1))-1:n];
        compute_cipher(this.x, this.k, this.y);
        this.next_x = y;
        $display("Key Schedule[%0d]: %h\n", i, key_schedule[i]);
        this.x = next_x;
        $display("Round key: %h", round_key);
        $display("-----\n");
    end
endtask

task sb_reset;
    key = trans.key;
    plaintext = trans.plaintext;
    round_key = 0;
    x = 0;
    next_x = 0;
    y = 0;
    k = 0;
    current_counter = 0;
    key_exp_in = 0;
    for(int i = 0; i < trans.num_round; i++) begin
        key_schedule[i] = 0;
    end
endtask

task main();
    repeat (no_transaction) begin
        mon2scb.get(trans);
        this.key = trans.key;
        this.plaintext = trans.plaintext;
        this.key_exp_in = key;
        $display("key= %h    plaintext = %h", trans.key, trans.plaintext);
        simon_coverage.sample();
        $display("-----Start Scoreboard-----");
        setup_keySchedule(this.round_key, this.k);
        x = plaintext;
        while(current_counter < trans.num_round) begin
            $display("Counter: %d", current_counter);
            key_exp_in = (current_counter < m)? key: {key_schedule[current_counter], key_schedule[current_counter-1],
                key_schedule[current_counter-2], key_schedule[current_counter-3]};
            compute_round_key(key_exp_in, (current_counter+1), round_key);
            $display("Round Key: %h", round_key);
            key_schedule[current_counter+1] = round_key;
            k = (current_counter < m)? key[(current_counter+1)*(n)-1:n] : key_schedule[current_counter];
            $display("Key Schedule: %h", key_schedule[current_counter]);
            compute_cipher(this.x, this.k, this.y);
            this.next_x = y;
            $display("X: %h \t K: %h \t Y: %h\n", this.x, this.k, this.next_x);
            this.x = next_x;
            current_counter = current_counter + 1;
        end
        if(this.x == trans.ciphertext)
            $display(" Matched Output !");
        else
            $display("Ciphertext Mismatch");
            $display("Plaintext : %d \t Key: %h", plaintext, key);
            $display(" Expected: %h \t Received(Monitor): %h\n", this.x, trans.ciphertext);
            sb_reset;
        end
    end
endtask
endclass

```

5.8 Coverage class

The coverage can be defined as a class or can be defined inside the scoreboard. The cover group comprises of a group of cover points that can be labelled as can be seen in Figure 8. The cover point can have a specific set of bins defined or illegal bins defined which specify the range of numbers, it can take.

Figure 13. Covergroup

```
covergroup simon_coverage;

    PLAINTEXT: coverpoint trans.plaintext {option.auto_bin_max = 64;}
    KEY: coverpoint trans.key {option.auto_bin_max = 64;}

endgroup
```

5.9 Top module

The top module in the testbench is the module comprising of the clocking and reset definition. The design instantiation is done inside the top module and the keywords to generate waveform are defined in the same.

Figure 14. Top module

```
//Top module
`timescale 1ns/1ps
`default_nettype none
module top;

    bit clk;

    simon_interface simon_if(clk);

    test testcase1(simon_if);

    simon DUT (
        .clk(simon_if.clk),
        .rst(simon_if.rst),
        .en(simon_if.en),
        .done(simon_if.done),
        .plaintext(simon_if.plaintext),
        .ciphertext(simon_if.ciphertext),
        .key(simon_if.key)
    );

    initial begin
        $vcdpluson;
        clk = 1'b0;
        forever #50ns clk=!clk;
    end
endmodule
```


5.10 Test program

The test is defined as a program where in the run task of the environment class is called.

The instance of test program is used in the top module.

Figure 15. Test

```
program test (simon_interface simon_if);  
  
    localparam loop = 2;  
  
    environment env;  
    simon_driver drv;  
  
    initial begin  
        $display("-----[Start Test]-----");  
        env = new (simon_if);  
        //env.reset;  
        env.build;  
  
        #1000;  
        env.run;  
        @(posedge simon_if.clk);  
  
    end  
  
endprogram
```

6. Simulation Result

Figure 15 shows the output of the simulator on VCS server. The display message “Matched Output” supports the self-checking part and shows that the expected output and the achieved output are same. The waveform in figure 12 show the iteration of round function and a ciphertext for the respective plaintext generated using an open-source simulator EDA Playground.

Figure 16. Simulation output

```
Counter: 27
Round Key: 0f12
Key Schedule: 801c
X: 33c70dfe      K: 801c      Y: 05fc33c7

Counter: 28
Round Key: c304
Key Schedule: 0f12
X: 05fc33c7      K: 0f12      Y: 232505fc

Counter: 29
Round Key: e409
Key Schedule: c304
X: 232505fc      K: c304      Y: 4e6e2325

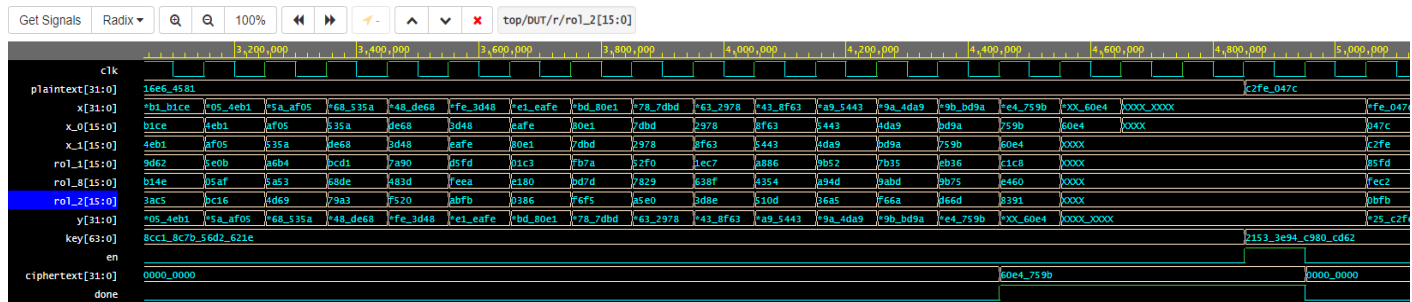
Counter: 30
Round Key: d5ba
Key Schedule: e409
X: 4e6e2325      K: e409      Y: f2d94e6e

Counter: 31
Round Key: a585
Key Schedule: d5ba
X: f2d94e6e      K: d5ba      Y: 9101f2d9

Matched Output !
Plaintext : 827678443      Key: e76b9cf900195e81
Expected: 9101f2d9      Received(Monitor): 9101f2d9

$finish at simulation time      1103050000
      V C S   S i m u l a t i o n   R e p o r t
Time: 1103050000 ps
CPU Time:      0.790 seconds;      Data structure size:      0.0Mb
Wed Dec 8 15:08:59 2021
[engr850-15@hafez ~/Simon_Cipher]$ urg -dir simv.vdb
```

Figure 17. Simulation waveform



6.1 Functional Coverage and Coverage Results

The coverage reports for the testbench are generated using the Synopsys tool on the VCS server. The command used is: `urg -dir simv.vdb`

Figure 18. Coverage Report for 64 bins

SYNOPSYS

Group : \$unit::scoreboard#(16,4)::simon_coverage

dashboard | hierarchy | modlist | groups | tests | asserts

Group : \$unit::scoreboard#(16,4)::simon_coverage

SCORE	WEIGHT	GOAL	AT LEAST	AUTO BIN MAX	PRINT MISSING
100.00	1	100	1	64	64

Source File(s) :

/courses/engr850/engr850-15/Simon_Cipher/scoreboard.sv

Summary for Group \$unit::scoreboard#(16,4)::simon_coverage

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
Variables	128	0	128	100.00

Variables for Group \$unit::scoreboard#(16,4)::simon_coverage

VARIABLE	EXPECTED	UNCOVERED	COVERED	PERCENT	GOAL	WEIGHT
PLAINTEXT	64	0	64	100.00	100	1
KEY	64	0	64	100.00	100	1

Variable : PLAINTEXT Variable : KEY

Summary for Variable PLAINTEXT

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
Automatically Generated Bins	64	0	64	100.00

Automatically Generated Bins for PLAINTEXT

NAME	COUNT	AT LEAST
auto[0:67108863]	7	1
auto[67108864:134217727]	6	1
auto[134217728:201326591]	9	1
auto[201326592:268435455]	5	1
auto[268435456:335544319]	4	1
auto[335544320:402653183]	4	1
auto[402653184:469762047]	3	1
auto[469762048:536870911]	5	1
auto[536870912:603979775]	5	1
auto[603979776:671088639]	4	1
auto[671088640:738197503]	5	1
auto[738197504:805306367]	3	1
auto[805306368:872415231]	5	1
auto[872415232:939524095]	5	1
auto[939524096:1006632959]	4	1
auto[1006632960:1073741823]	5	1
auto[1073741824:1140850687]	4	1
auto[1140850688:1207959551]	4	1
auto[1207959552:1275068415]	2	1
auto[1275068416:1342177279]	10	1
auto[1342177280:1409286143]	4	1
auto[1409286144:1476395007]	6	1
auto[1476395008:1543503871]	4	1
auto[1543503872:1610612735]	6	1
auto[1610612736:1677721599]	3	1
auto[1677721600:1744830463]	7	1
auto[1744830464:1811939327]	4	1

Figure 19. Coverage Report for 128 bins



Group : \$unit::scoreboard#(16,4)::simon_coverage

[dashboard](#) | [hierarchy](#) | [modlist](#) | [groups](#) | [tests](#) | [asserts](#)

Group : \$unit::scoreboard#(16,4)::simon_coverage

SCORE	WEIGHT	GOAL	AT LEAST	AUTO BIN MAX	PRINT MISSING
100.00	1	100	1	64	64

Source File(s) :

/courses/engr850/engr850-15/Simon_Cipher/scoreboard.sv

Summary for Group \$unit::scoreboard#(16,4)::simon_coverage

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
Variables	256	0	256	100.00

Variables for Group \$unit::scoreboard#(16,4)::simon_coverage

VARIABLE	EXPECTED	UNCOVERED	COVERED	PERCENT	GOAL	WEIGHT
PLAINTEXT	128	0	128	100.00	100	1
KEY	128	0	128	100.00	100	1

Variable : PLAINTEXT > Variable : KEY >

Summary for Variable PLAINTEXT

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
Automatically Generated Bins	128	0	128	100.00

Automatically Generated Bins for PLAINTEXT

Bins

NAME	COUNT	AT LEAST
auto[0:33554431]	6	1
auto[33554432:67108863]	7	1
auto[67108864:100663295]	8	1
auto[100663296:134217727]	2	1
auto[134217728:167772159]	3	1
auto[167772160:201326591]	10	1
auto[201326592:234881023]	6	1
auto[234881024:268435455]	4	1
auto[268435456:301989887]	7	1
auto[301989888:335544319]	4	1
auto[335544320:369098751]	3	1
auto[369098752:402653183]	5	1
auto[402653184:436207615]	8	1
auto[436207616:469762047]	3	1
auto[469762048:503316479]	5	1
auto[503316480:536870911]	4	1
auto[536870912:570425343]	3	1
auto[570425344:603979775]	8	1
auto[603979776:637534207]	3	1
auto[637534208:671088639]	6	1
auto[671088640:704643071]	2	1
auto[704643072:738197503]	8	1
auto[738197504:771751935]	6	1
auto[771751936:805306367]	6	1
auto[805306368:838860799]	8	1
auto[838860800:872415231]	1	1
auto[872415232:905969663]	6	1
auto[905969664:939524095]	6	1
auto[939524096:973078527]	4	1

Figure 20. Coverage Report for 512 bins

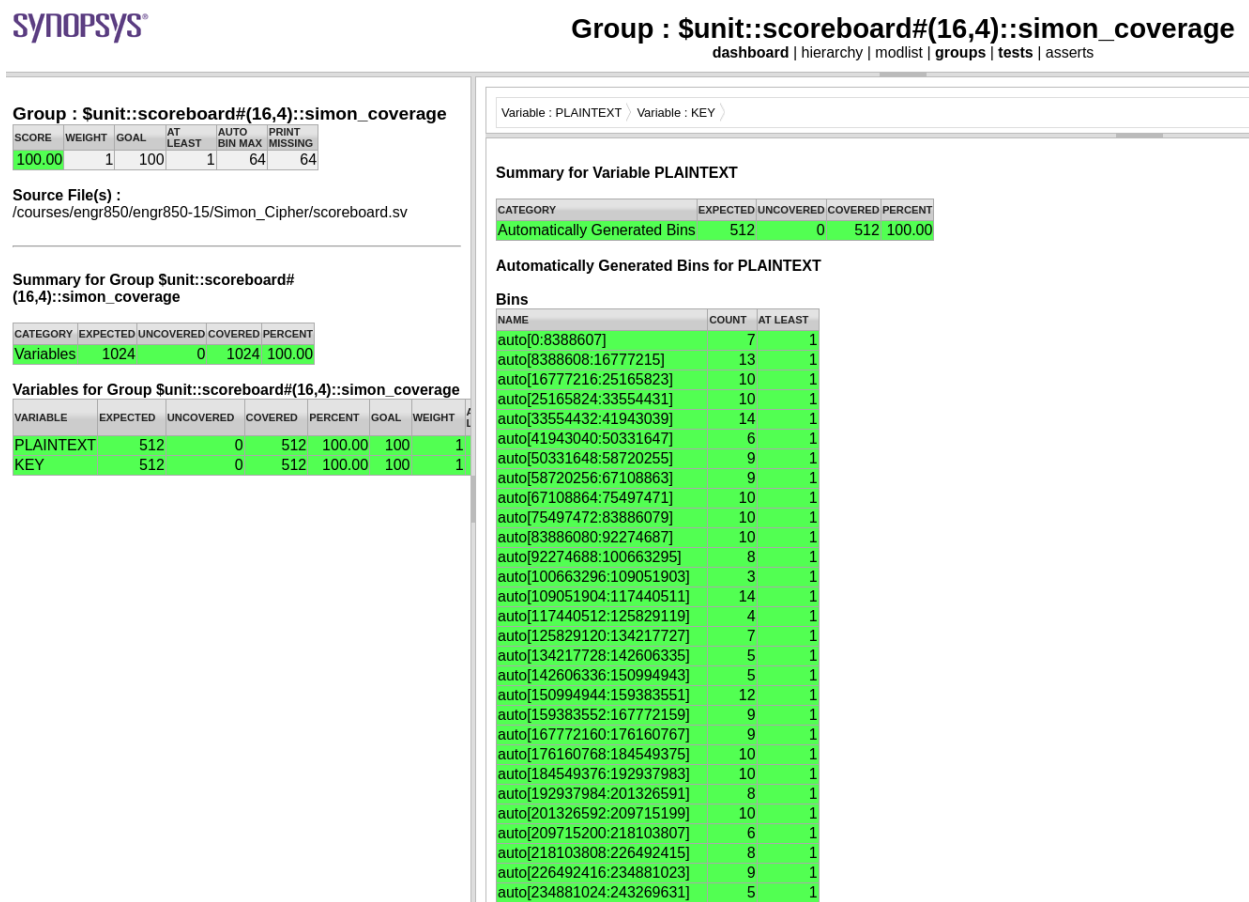


Figure 21. Coverage Report for 1024 bins

SYNOPSIS

Group : \$unit::scoreboard#(16,4)::simon_coverage

dashboard | hierarchy | modlist | groups | tests | asserts

Group : \$unit::scoreboard#(16,4)::simon_coverage

SCORE	WEIGHT	GOAL	AT LEAST	AUTO BIN MAX	PRINT MISSING
100.00	1	100	1	64	64

Source File(s) :

/courses/engr850/engr850-15/Simon_Cipher/scoreboard.sv

Summary for Group \$unit::scoreboard#(16,4)::simon_coverage

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
Variables	2048	0	2048	100.00

Variables for Group \$unit::scoreboard#(16,4)::simon_coverage

VARIABLE	EXPECTED	UNCOVERED	COVERED	PERCENT	GOAL	WEIGHT
PLAINTEXT	1024	0	1024	100.00	100	1
KEY	1024	0	1024	100.00	100	1

Variable : PLAINTEXT Variable : KEY

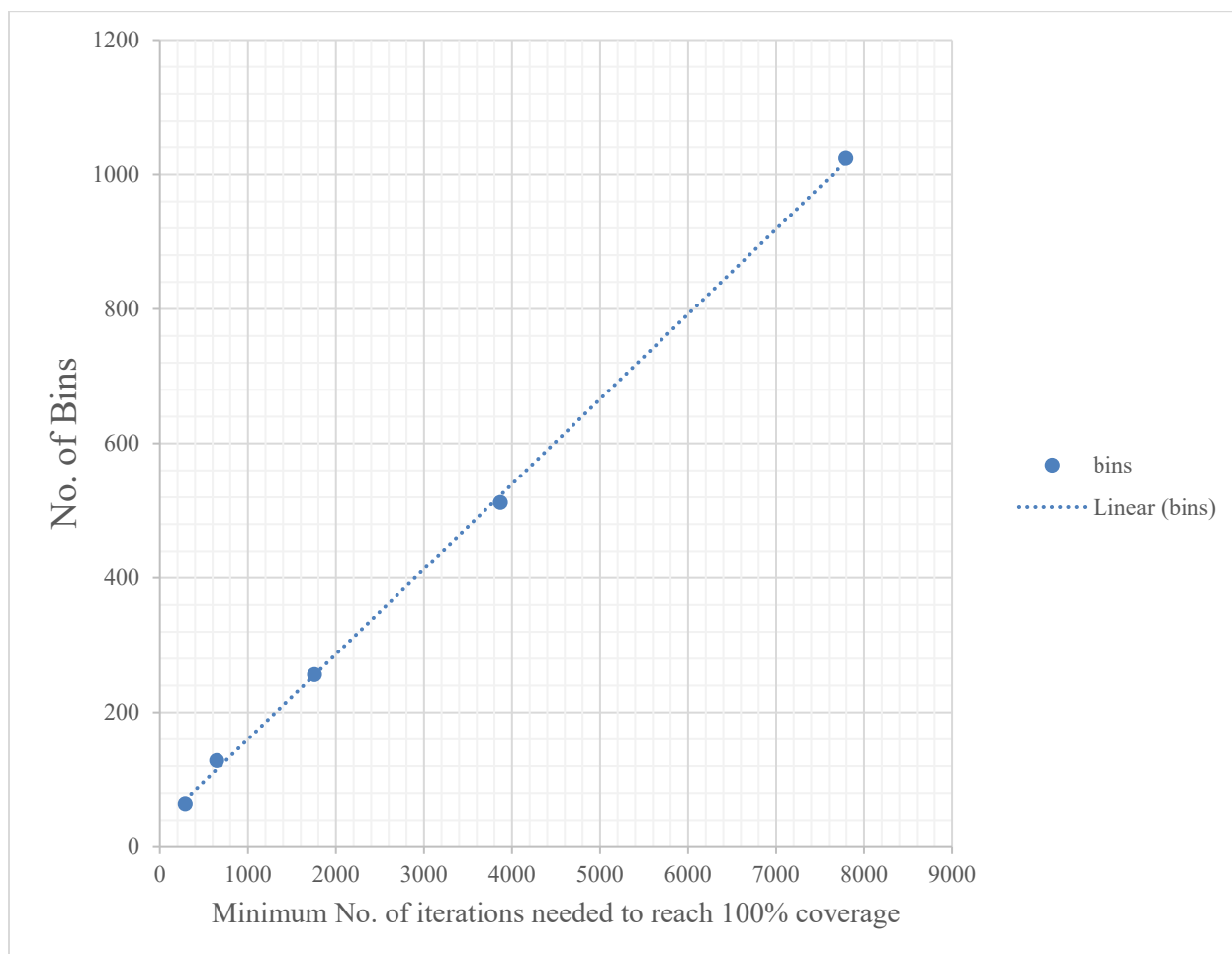
Summary for Variable PLAINTEXT

CATEGORY	EXPECTED	UNCOVERED	COVERED	PERCENT
Automatically Generated Bins	1024	0	1024	100.00

Automatically Generated Bins for PLAINTEXT

NAME	COUNT	AT LEAST
auto[0:4194303]	5	1
auto[4194304:8388607]	14	1
auto[8388608:12582911]	10	1
auto[12582912:16777215]	9	1
auto[16777216:20971519]	9	1
auto[20971520:25165823]	14	1
auto[25165824:29360127]	11	1
auto[29360128:33554431]	6	1
auto[33554432:37748735]	9	1
auto[37748736:41943039]	11	1
auto[41943040:46137343]	6	1
auto[46137344:50331647]	5	1
auto[50331648:54525951]	8	1
auto[54525952:58720255]	9	1
auto[58720256:62914559]	7	1
auto[62914560:67108863]	8	1
auto[67108864:71303167]	11	1
auto[71303168:75497471]	9	1
auto[75497472:79691775]	7	1
auto[79691776:83886079]	11	1
auto[83886080:88080383]	10	1
auto[88080384:92274687]	7	1
auto[92274688:96468991]	7	1
auto[96468992:100663295]	7	1
auto[100663296:104857599]	8	1
auto[104857600:109051903]	7	1
auto[109051904:113246207]	11	1
auto[113246208:117440511]	13	1

The figure 21 shows a comparison plot between minimum number of rounds required to achieve 100% coverage as the value of option_auto.bin.max is increased from 64 up to 1024.

Figure 22. Coverage report Comparison

Conclusion

SIMON cipher is a flexible algorithm designed to be compatible with different block sizes to increase the use of encryption in situations where the systems are constrained. In this paper, class based functional verification of SIMON_{32/64} algorithm is performed. Simon _{32/64} has a small value of data length and key, therefore being the best suitable option for lightweight cryptography. The coverage report is achieved 100% for all maximum value of bins starting from default value: 64 to 1024, supporting the functional verification flexibility for any random value of key and plaintext respective to the size of n and m for Simon_{2n/nm}. The scoreboard class comprises of a self-checking module which compares encrypted testbench output with the design output.

References

- [1] Mourouzis, T., Song, G., Courtois, N., & Christofii, M. (2015). Advanced differential cryptanalysis of reduced-round SIMON64/128 using large-round statistical distinguishers.
- [2] Beaulieu, Shors, Smith, Clark, Weeks & Wingers (2013). The Simon and Speck Families of Lightweight Block Ciphers. *National Security Agency*
- [3] Courtois, N., Mourouzis, T., Song, G., Sepehrdad, P., & Susil, P. (2014, August). Combined algebraic and truncated differential cryptanalysis on reduced-round simon. In *2014 11th International Conference on Security and Cryptography (SECRYPT)* (pp. 1-6). IEEE.
- [4] Sawant, Kamthe, Shaha, Morajkar & Sakpal (2019). Implementation of SIMON & SPECK Algorithm. *Journal of Emerging Technologies and Innovative Research*
- [5] HEADS-UNCC/SimonCipherVerilog. GitHub- SimonCipherVerilog.
<https://github.com/HEADS-UNCC/SimonCipherVerilog>
- [6] Rashidi, B. (2019). High-throughput and flexible ASIC implementations of SIMON and SPECK lightweight block ciphers. *International journal of circuit theory and applications*, 47(8), 1254-1268.
- [7] El Hadj Youssef, W., Abdelli, A., Dridi, F., & Machhout, M. (2020). Hardware Implementation of Secure Lightweight Cryptographic Designs for IoT Applications. *Security and Communication Networks*, 2020.
- [8] Dar, A. B., Lone, M. J., & Hussain, N. (2021). Revisiting Lightweight Block Ciphers: Review, Taxonomy and Future directions. *Computer Science Review, Forthcoming*.
- [9] <https://www.chipverify.com/systemverilog/systemverilog-tutorial>, n.d.

- [10] <https://verificationguide.com/systemverilog/systemverilog-tutorial/>, 2016-2020